A Configurable Framework for Multi-Scenario Industrial Agent Integration

Anonymous Authors

***Abstract*—Industrial-agent systems must connect collaborative reasoning to production context and heterogeneous equipment while preserving process-specific operating rules. This paper presents a configurable framework for that integration. Its architecture combines agent-team collaboration and configurable execution harnesses with line–product–recipe–run context, analysis and modeling services, semantic data-acquisition and write-control nodes, and protocol drivers. Scene blueprints bind signal maps, objectives, and guards; channel profiles, tool policies, and per-agent bindings scope the available operations. An agent or caller invokes a named node-exploration or recipe workflow, whose runtime checks and evidence lifecycles are distinct. A historical seed-42 campaign records terminal-check success in four configured simulator scenes. Separate protocol tests covered five acquisition families and four writable mappings, while nine deployment-suite runs passed 782 deterministic assertions. Live-engine and controller archives, plus a cadence-overridden recipe case, exercise complementary workflows; the recipe case saved but did not apply its accepted version. The evidence supports integration in the tested simulator settings. Because the four-scene records use a historical direct- node path, they do not validate current profile and binding configurations across scenes, physical transfer, or comparative control benefit.**

***Index Terms*—Industrial agents, industrial communication, industrial informatics, production systems, semantic data integration.**

I. Introduction

LARGE language-model agents have been applied to plant-facing tasks, including operator-reviewed control-sequence generation, module-level factory skills, and production scheduling [1]–[3]. These studies demonstrate the potential of agents in industrial workflows, but address different tasks and integration assumptions. Across plants, agent roles, signals, protocols, objectives, and production context also vary. Within this selected work, no study evaluates a shared framework that combines configurable agent collaboration and execution harnesses, production context, heterogeneous process input/output, and route-specific operations across scenes.

Consider injection molding: an agent team may use recent measurements and prior process knowledge to diagnose insufficient packing and propose a holding-pressure change. The framework must resolve the active line, product, recipe, and run; map the proposed quantity to a device interface; invoke an operation permitted for that context; and link command evidence to the resulting quality observations. A different process may use another signal map, protocol, or target while retaining shared services. In each case, the proposal, device setting, and process response remain distinct and tied to their production context.

This paper asks how a reusable industrial-agent framework can connect collaborative reasoning to production context and heterogeneous process I/O while retaining a traceable path from candidate operation to process observation. The framework separates harness adaptation, scene configuration, process analysis, and heterogeneous process I/O. Scene blueprints supply signal maps, objectives, and guards; channel profiles, tool policies, and per-agent bindings scope the available operations. An agent or caller invokes a named operation, and shared runtime gates validate it before dispatch; the framework does not autonomously choose an execution route. Figure 1 shows the software layers, and Fig. 2 traces the alternative operation lifecycles.

This paper makes three contributions:

1) It presents an extensible architecture that combines agent-team orchestration and configurable execution harnesses with analysis and model services, production context, semantic industrial I/O, and typed extension interfaces.
2) It specifies how scene blueprints, signal and protocol mappings, channel profiles, tool policies, and per-agent bindings configure shared services, while node exploration and recipe operations retain distinct execution and evidence lifecycles.
3) It reports historical outcomes from four configured simulator scenes and evaluates separate protocol, live-engine, deployment-suite, and recipe-path evidence. These archives do not form a paired strategy comparison or a single current-version end-to-end trial.

The remainder reviews related industrial-agent and integration work, describes the framework and its operational routes, reports the evaluation, and discusses the evidence scope and limitations.

II. Related Work

This section situates the framework among industrial-agent systems and the production models and interfaces used in plant integration. The comparison below identifies a gap only within the selected studies in Table I.

A. Industrial Agents and Plant Integration

Industrial-agent studies span service-oriented supervision and programmable-logic-controller (PLC) or closed-loop control workflows [4]–[6], resource coordination and digital-twin-based fault handling [7]–[9], and natural-language machine

TABLE I
SELECTED PLANT-FACING AGENT SYSTEMS AND THEIR INTEGRATION SETTINGS. EVALUATION EVIDENCE IS REPORTED WITHIN EACH SOURCE'S SCOPE AND IS NOT A DIRECT PERFORMANCE COMPARISON.

System	Application scope	Industrial integration	Source-reported evaluation
MADE [1]	Operator-reviewed control-sequence generation	Structured OPC Unified Architecture (OPC UA) commands and authorization	40 control tasks; virtual and physical demonstrations
Köhle et al. [2]	Module agents and factory skills	OPC UA and Message Queuing Telemetry Transport (MQTT) with component-state tool availability	State cross-verification in a simulated six-module plant
Waseem et al. [3]	LLM-based robot-assignment scheduling	Prompt-constrained controller for a simulated line	Scheduling throughput
Proposed framework	Collaborative tasks, configurable agent harnesses, and analysis/model services	Production context and semantic process nodes over multiple protocol mappings	Historical four-scene simulator archive; separately archived protocol, software-suite, and recipe-route evidence

interfaces and line controllers [3], [10]. These systems establish plant-facing applications for agents, but differ in task, interface, and production assumptions.

Table I compares three representative settings: reviewed control-sequence generation, module-scoped skills, and line scheduling. It is not an exhaustive nearest-work set; the gap below is limited to these comparators, while the other cited studies provide context for adjacent integration themes.

The framework treats authority and confirmation as operational services, consistent with safety-aware decision support [11]; it does not replace process protection or establish operator effectiveness.

B. Production Context, Agent Interfaces, and Automation

The Model Context Protocol (MCP) standardizes tool discovery and calls with request-scoped authorization [12]; ISA-88 and ISA-95 provide concepts for recipes and production operations [13], [14]. Run-to-run methods connect process observations to successive production runs [15]. At the agent level, AgentSpec and CaMeL address tool behavior and capability-based information flow [16], [17]. These works provide complementary abstractions for tool access, production context, and data flow; they do not by themselves specify a shared integration architecture across the heterogeneous process routes studied here.

Industrial security and record requirements provide further design constraints: FDA guidance discusses trustworthy records and audit trails [18], IEC 62443-3-3 specifies system-security requirements [19], and DNP3 includes select-before-operate [20]. Control-assurance methods address state evolution using shielding, control-barrier functions, or Simplex architectures [21]–[23]. Taken together, the three representative prior systems in Table I do not evaluate a configurable composition of agent harnesses, production context, semantic process I/O, and distinct node and recipe workflows across scenes. This bounded gap motivates an integration architecture and implementation; it is not a claim that the wider literature lacks these capabilities, nor a claim of a new controller or safety method.

III. SYSTEM DESIGN AND OPERATION

The design goal is to adapt agent-assisted work across industrial scenes without rewriting the shared runtime for each line. The framework separates five concerns: agent-harness adaptation, production-context configuration, evidence and analysis services, tool exposure and execution checks, and protocol-facing process I/O. This separation keeps an agent's proposal, an endpoint's command receipt, and the measured process response as distinct objects. Scene variation is expressed through context, signal maps, bindings, and drivers; shared services implement the interaction contract.

A. Software Architecture and Configuration

Figure 1 maps the implementation into these layers. A harness-neutral agent interface and registry allow channel members to run through registered engine adapters. Runtime wiring supplies each member with its channel task, shared team context, scenario guidance, optional line brief, profile, and workspace. The tool catalog is then scoped by role, channel profile, tool policy, and per-agent bindings. When a named tool is invoked, the shared dispatch path rechecks invocation scope, channel policy, profile and mode, line grants, and declared resource references before calling the industrial service.

The production context is the active line–product–recipe–run tuple. Scene blueprints bind devices and signal maps to process objectives and guards, while line, product, and recipe definitions remain separately versioned from a running batch. Per-agent bindings restrict readable DAQ nodes, writable controls, or recipe operations to the resources configured for that member. SDK, REST, MCP, and A2A interfaces extend the service surface; the CLI and console provide user access. These configuration layers shape available tools and context.

B. Evidence Services and Semantic Process I/O

DAQ nodes represent typed observations with engineering units, timestamps, and quality metadata; DCW nodes represent writable quantities with engineering ranges, transforms, and driver configuration. Driver registries resolve built-in and plugin implementations for Modbus TCP/RTU-over-TCP, OPC UA, MQTT, HTTP/REST, and MES mappings. This semantic layer lets an operation name a process quantity while the driver handles its endpoint representation. DAQ ingestion and storage are separate from the execution clock, so queries and exports can use retained time-series data.

Analysis and model services provide evidence for candidate formation. Diagnostics use aligned DAQ exports; retrieval

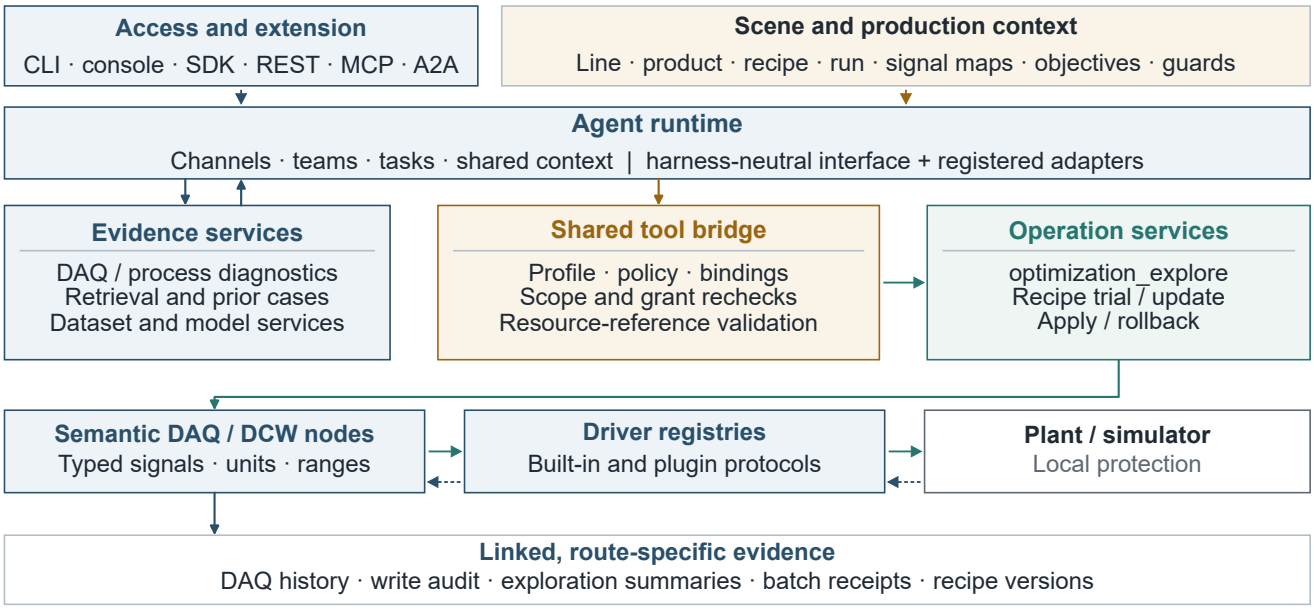


Fig. 1. Software architecture and industrial boundary. Registered harness adapters run channel agents; scene context, profiles, policies, and bindings shape the tool surface. An agent invokes a named operation through a shared dispatch gate, which calls the applicable service. Semantic DAQ/DCW nodes resolve built-in or plugin drivers. Command receipts, DAQ history, and route-specific records remain linked to production context; local plant protection is outside the framework.

supplies process knowledge and prior cases; optional dataset and model workflows prepare evaluated candidates. These services return evidence to the agent but do not grant write authority. A recommendation may therefore inform an operation without changing the tool surface or bypassing dispatch-time checks. The experiments do not evaluate model-assisted control response.

C. Task-to-Operation Procedure

Figure 2 shows the operational procedure. First, a task is bound to its production context and the agent gathers available process evidence. Second, the agent or caller invokes a named operation from the configured tool surface; profiles and bindings constrain the available operations, but do not silently choose one. Third, shared dispatch and operation-specific services revalidate scope, authority, object bindings, and applicable execution conditions. Finally, the selected workflow returns its own command evidence and record type, while process observations are queried and evaluated separately.

The bounded node probe is exposed as `optimization_explore` to an AML-optimization profile. It validates a bound control and target DAQ scope, captures a baseline, invokes the internal DCW write path, waits, and samples a trailing target window before saving an exploration summary. A successful value-changing write can also open a node-optimization record. Driver-level setting confirmation and the later DAQ process response are separate evidence.

Recipe operations follow a different lifecycle. `recipe_trial` checks the bound recipe and active run, then dispatches candidate overrides as a batch with per-parameter verification. The caller subsequently queries DAQ and decides whether to save a new definition with `recipe_update`, restore one with `recipe_rollback`, or dispatch a version with `recipe_apply`. These steps are separate: a trial does not save or apply a version, and endpoint verification is not a process-response measurement. Public direct-node control is closed; the bounded probe and recipe operations are the configured operation surfaces. The historical four-scene results use an earlier direct-node path and do not test current profile or binding adaptation across scenes.

D. Admission, Timing, and Approval

A tool invocation is admitted against its operation, actor, resource bindings, and active production context. For an online node intervention, the admissible value interval is

$$\mathcal{I}_n^{\text{online}}(t) = \mathcal{H}_n \cap \mathcal{B}_n \cap \mathcal{P}_n(t) \cap \mathcal{R}_n(t), \tag{1}$$

where $\mathcal{H}_n$ is the hard engineering range and $\mathcal{B}_n$, $\mathcal{P}_n$, and $\mathcal{R}_n$ are baseline, active-product, and active-recipe limits. Missing optional limits add no restriction; an empty intersection refuses the value. The excluding layer is returned for interpretation (Fig. 3a). Recipe dispatch instead checks $\mathcal{H}_n \cap \mathcal{B}_n \cap \mathcal{P}_n(t)$, omitting the previous recipe window while applying its replacement. Admission constrains setpoint magnitude; process objectives and coupled variable guards are evaluated from later observations.

Timing guards are route-specific. Online node writes enforce an effective minimum spacing of 60 s. Recipe dispatch uses a per-recipe interval anchored to approval, while trials also use a configurable per-line cadence (default 300 s). The recipe interval can refuse a proposal before an approval request is created. Checks occur at proposal or execution according to the operation, and experiment-specific pacing overrides are reported with the corresponding results.

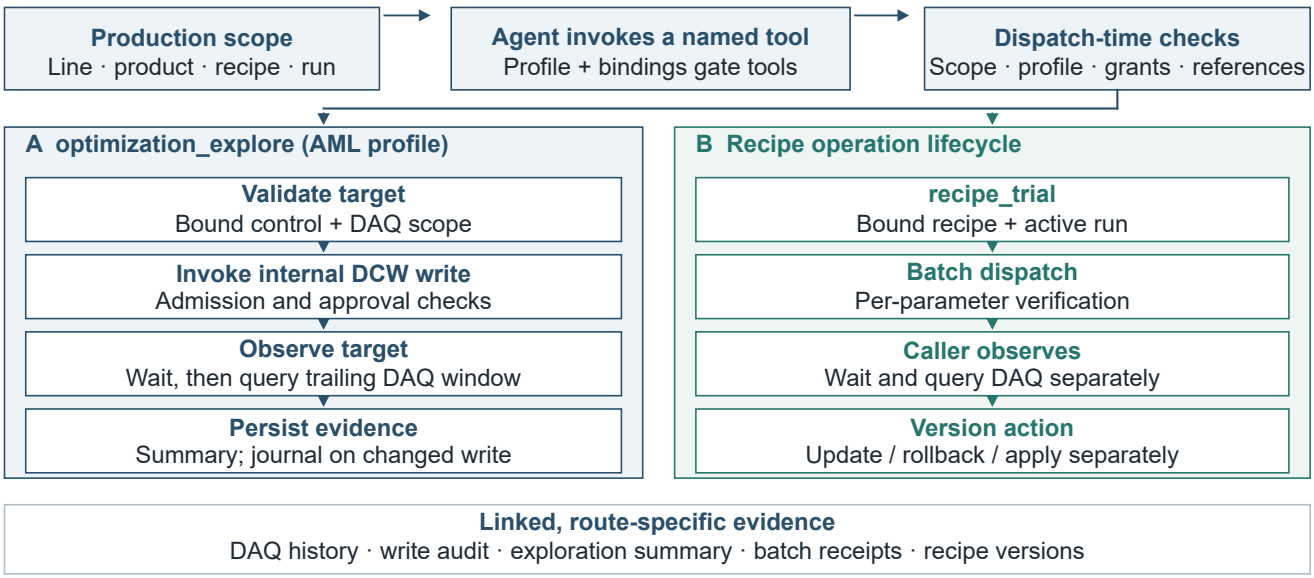


Fig. 2. Agent-invoked operation flow. Shared context and evidence inform a candidate; the configured tool surface and runtime gates constrain its execution. Lane A is the AML-profile-gated `optimization_explore` probe, with a DCW write followed by a target query and exploration summary. Lane B is the recipe workflow, where a trial returns batch receipts before the caller queries DAQ; version update, rollback, and apply are separate operations. The lanes are alternatives, not an automatic route choice.

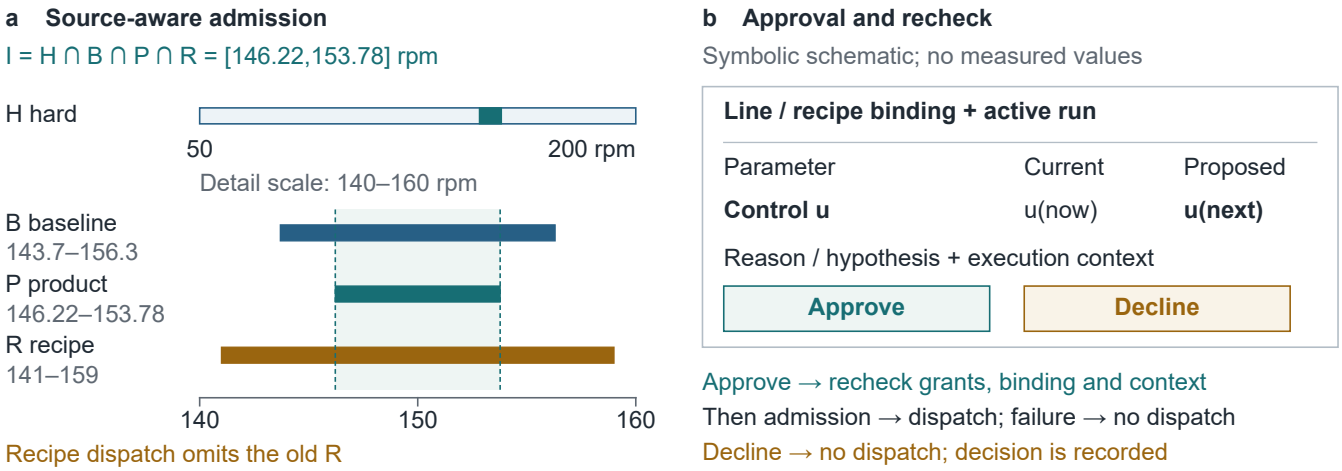


Fig. 3. Admission and approval boundaries. (a) Online node interventions use the four-layer value interval; recipe dispatch omits the prior recipe window. (b) Human approval binds a proposed action and authority fields; execution checks run again after approval, which does not reserve equipment state.

Manual bindings create an approval request containing the actor, node or recipe, candidate values, applicable limits, expiry, and parameter bases. The default wait expires after 180 s and refuses execution. After approval, the executor rechecks authorization and active limits to account for context changes during the wait. For `recipe_apply`, a changed definition requires a new proposal; an unchanged approval dispatches a copy of the approved parameter snapshot. A line-level recipe mode defaults to manual; changing it to automatic requires administrator confirmation. The approval record authorizes a workflow action and does not replace process protection.

E. Command Verification and Process Evidence

Command confirmation and process response answer different questions. A driver ACK is graded as `readback-verified`, `transport-ack`, or `unverified`. A matching endpoint read supports the first grade; a transport or MES application receipt supports the second. Recipe batch verification assigns `verified`, `unverified`, or `failed`; if no matching readback is available, it attempts up to three independent reads, 2 s apart. A matching retry upgrades the final ACK, while exhausted retries remain unverified. The initial ACK remains in write history and the final ACK and verdict are retained with the batch. Per-parameter receipts preserve successful outcomes when another parameter fails. A readback-verified setpoint does not establish target attainment.

The node-probe workflow captures a starting control and target baseline, then waits 10–300 s (default 45 s) and queries a trailing 45-s target window. Since each window is relative to

its query time, approval can separate baseline sampling from actuation; a 10-s wait can leave about 35 s of pre-write samples in the later window. The command receipt and target summary are therefore returned separately. A successful changed-value write opens a node-optimization record; the probe saves its exploration summary after the target query. Refusal or expired approval creates a rejected summary without a post-write sample.

The caller performs recipe-path observation after batch dispatch. The October recipe case therefore compares a recipe definition, endpoint readback, and DAQ sample as separate representations. Saving a definition, applying it to equipment, and observing its process response are separately evidenced operations. The evaluation does not combine these route-specific records into one universal end-to-end record.

An open node record provides a monitoring backstop. After 120 s it polls every 30 s; three cumulative samples outside the recipe monitoring window trigger a system-attributed rollback request toward the preceding value. Missing samples do not count as breaches. A caller's `keep`, `rollback`, or `uncertain` verdict records its interpretation; compensation is a new command subject to its applicable admission policy. The write audit deduplicates the same value on the same node within 10 s, while batch receipts, exploration summaries, DAQ history, and recipe versions retain their distinct lifecycles. Linking these records by actor and production context allows the command, endpoint evidence, and process observation to be inspected without treating them as interchangeable.

IV. Experimental Evaluation

The evaluation reports complementary evidence for shared collaboration, production-context, and industrial-I/O services across process scenes and operational routes. Q4 reports four configured simulator scenes; Q1 tests acquisition and policy-checked writes over multiple protocol mappings. Q2 records two live-engine tasks under the same harness, provider, and model setup, while Q3 follows one deterministic controller over three seeds. Q5 assesses deployment-suite behavior, and Q6 examines a recipe-path lifecycle. All process experiments use repository simulators. These evidence groups are complementary archives, not one joint current-version trial or a paired strategy comparison; their identifiers and source mappings are given in the reproducibility notes.

A. Evidence Units and Experimental Procedure

Q1–Q4 use the seed-42 integrated archive (R1) and its raw scenario records; the admission ablation uses a separate companion run (R2). R1 can be audited, but its historical runtime worktree is not archived, so the recorded source commit does not guarantee exact replay. These historical missions use the direct-node interface and scripted proposal policies, apart from the two Q2 live-engine tasks. Q3 additionally uses seeds 43–44; Q5 uses nine deployment-suite executions, and Q6 uses the October 18 recipe archive. The cast-film reference preset uses an author-defined objective J combining thickness conformance, defects, energy, and throughput.

Separately, 65 channel/profile and composition checks passed in an October 20 archive and an October 24 rerun with temporary storage and mock I/O. They cover approval outcomes, starting-control capture, and approved-recipe snapshot consistency; they are not process-level cross-scene trials. For each Q4 mission, the archive retains the configured scene, initial controls, proposed writes, available command evidence, post-wait DAQ observations, and terminal guard result. Each process contributes one seed-42 trajectory; writes within a trajectory are sequential observations, not independent trials.

The deployment regression spans 20 phases, from commissioning and protocol I/O to agent missions, controller reuse, and concurrent processes. Its author-defined checks report rejected probes, per-parameter confirmation, recipe/readback/mirror agreement, process-target margins, and event spans. The nine sequential runs and 782 deterministic assertions are summarized in Table III; assertions are software checks, not plant samples. Because each protocol line has six writes, latency is reported as the archived upper median and observed maximum, not as a tail percentile.

B. Historical Four-Process Simulator Missions (Q4)

The concurrent campaign provides one seed-42 trajectory per process. Figure 4 pairs archived manipulated controls u with the corresponding response y rather than showing response alone. All four terminal attainment checks passed; two lay near a target boundary.

Injection molding used two changes: hold pressure $45 \rightarrow 58.2$ bar and hold time $6 \rightarrow 10.64$ s. Part weight rose from 31.25 to 32.17 g for a 32.5 ± 0.35 g window, only 0.02 g above its lower limit. Biaxially oriented polyethylene terephthalate (BOPET) changed casting speed $32 \rightarrow 33.8$ m/min, fast-roll speed $118 \rightarrow 120.3$ m/min, and rail width $3000 \rightarrow 3035$ mm. Thickness fell from 28.10 to 25.66 μm for a 25.0 ± 0.7 μm target, 0.04 μm inside the upper limit. Its three changes each have a recorded keep verdict and journal identifier.

Wastewater treatment required 15 writes across several nodes. Dissolved oxygen rose from 0.21 to 3.50 mg/L, within 3.4 ± 0.6 mg/L. Some admissible blower proposals reduced compliance margin and were reverted. Terminal guards remained within their declared limits, although pH moved toward its lower bound (6.59 to 6.38; limit 6.0). Dissolved oxygen, total phosphorus (TP), and pH are different quantities and are not concatenated into one output trajectory.

Annealing combined furnace-temperature changes with line-speed increases from 140 to 164 m/min. The recorded Vickers hardness (HV) trajectory moved from 134.52 to 99.84 HV; the separate terminal measurement was 100.08 HV, within 95 ± 6 HV. A 176 m/min attempt produced 101.3 HV and was reverted. The first temperature step has no optimization record identifier in the archive, which bounds record-level traceability. These historical missions document process behavior under a shared scripted task scaffold and retain the distinction between a candidate command, a reverted candidate, and a terminal observation. They do not evaluate current profile or per-agent binding variation across scenes.

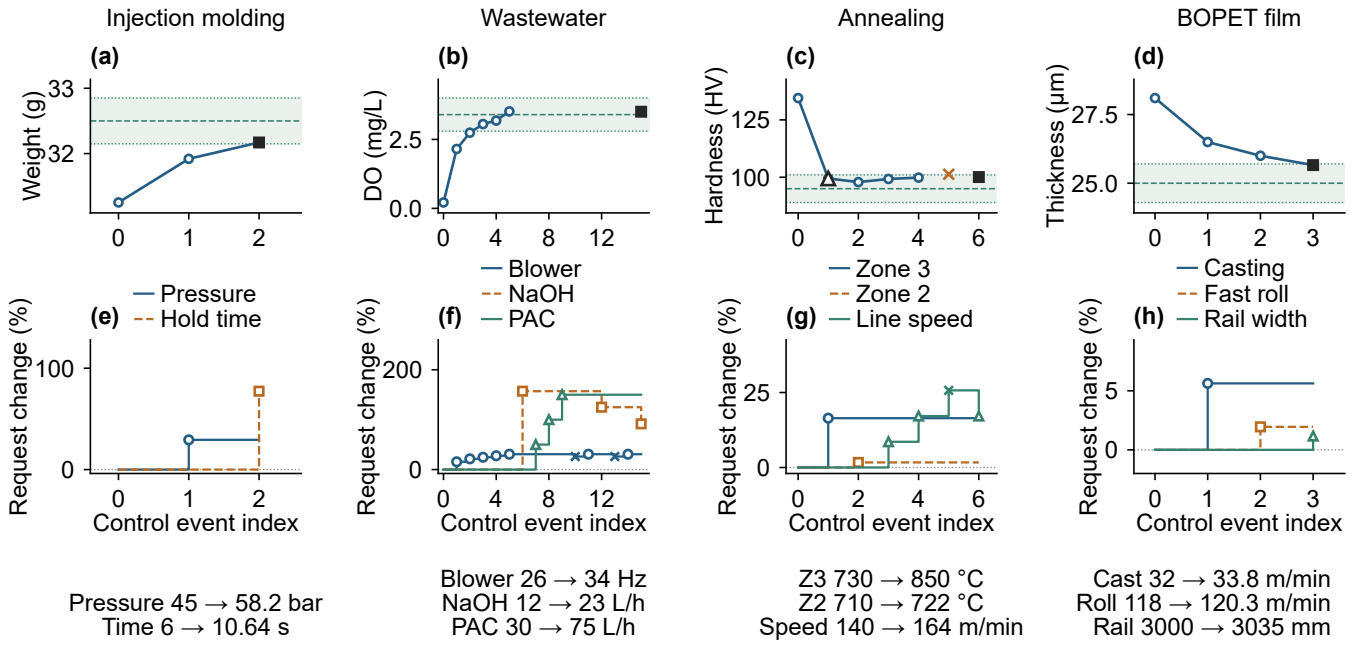


Fig. 4. Historical scripted missions (seed 42, one trajectory per process). (a–d) Weight, dissolved oxygen (DO), hardness, and film thickness with their target windows. (e–h) Requested controls as percentage change from native baselines: injection pressure 45 bar/time 6 s; wastewater blower 26 Hz/NaOH 12 L/h/PAC 30 L/h; annealing zone 3 730°C/zone 2 710°C/speed 140 m/min; BOPET casting 32 m/min/fast roll 118 m/min/rail 3000 mm. Lines connect event indices, not physical time; unchanged requested values carry forward. Crosses mark reverted candidates and squares distinct terminal observations. DO excludes pH/TP events; annealing's first step lacks an optimization record.

C. Heterogeneous Protocol Integration and Route Checks (Q1)

The suite acquired 55 samples over five protocol families and issued policy-checked writes over four; the RTU-over-TCP line was acquisition-only. Archived upper-median write latencies ranged from 19.3–47.8 ms, with observed maxima 20.1–248.9 ms. The four writable lines accepted legal writes and rejected 24/24 out-of-window probes. A configuration-only second preset added 9/9 rejected probes on three further writable lines. These results exercise the same resolver against different signal maps.

Table II isolates admission from confirmation in a mock-driver API fixture. Each arm replayed six attack classes in three deterministic repetitions (18 attacks per arm); separate boundary, legal, and timed probes are excluded from that denominator. Removing the soft-window interlock allowed six soft-window probes to execute, while the hard-range probes remained refused. Archived outcomes and journal records link the refusals and executed commands to their configured arms. Removing readback did not alter admission. Confirmation can therefore check an execution outcome, but cannot replace a pre-write gate. A frozen-measurement drill further separated command and process evidence: the setpoint and readback agreed, yet the monitoring quantity remained outside its window and triggered backstop rollback after 130.2 s. A manual-binding drill suspended execution until harness-issued approval (461 ms mediation span).

D. Live-Engine Tasks and Deterministic Controller (Q2–Q3)

The archived team workflow linked a parent task, a bound worker, a command, a verdict, and its journal. On the single-node cast-film mission, the policy changed the heater from 200 to 204.704 °C; the observation settled at 204.70 °C within the declared target. The task reached `COMPLETED`. The BOPET mission expanded this workflow to nine devices and 49 signals, with three manipulated nodes (Section IV-B).

TABLE II
MOCK-DRIVER API ABLATION: SIX ATTACK CLASSES, THREE DETERMINISTIC PROPERTY-TEST REPETITIONS PER ARM (18 ATTACKS, EXCLUDING SEPARATE BOUNDARY PROBES). FOUR HARD-RANGE CLASSES ARE REFUSED IN EVERY ARM; TWO SOFT-WINDOW CLASSES CHANGE OUTCOME.

Arm	Soft window	Readback	Refused	Executed
full	✓	✓	18/18	0
no-readback	✓	—	18/18	0
no-interlock	—	✓	12/18	6
ungated	—	—	12/18	6

Two live-engine tasks used the same `omp` harness, Zhipu Coding Plan provider, and GLM-5.3-Flash model on a scratch line; they are execution traces, not a strategy comparison. The archive records a bounded write, a goal-driven descent to 51.7 μm for a $52.0 \pm 0.8\,\mu$m target, and a recipe save. The original contemporaneous omp logs and checked run outcomes trace proposal, policy-checked action, observations, and retained recipe. The reported 51.7 μm is the run-summary terminal value; the log's 52.02 μm window mean is a different aggregation.

A deterministic controller used the dimensionless cast-film objective

$$J = 55J_{\text{thick}} + 25J_{\text{quality}} + 8J_{\text{energy}} + 7J_{\text{throughput}}, \tag{2}$$

whose components score thickness conformance, product qual-

TABLE III
DEPLOYMENT-SUITE ASSERTIONS ACROSS NINE SEQUENTIAL EXECUTIONS; CHECKS ARE DETERMINISTIC ASSERTIONS, NOT INDEPENDENT PLANT SAMPLES.

Pass	Test surface	Runs	Checks
1	baseline S0–S7	3	237
2	+ grading and master switch	3	273
3	hardened regression surface	3	272
Total	zero failed checks	9	782

ity, and control-based energy and throughput proxies. The latter two terms use screw speed and line speed, rather than independently metered energy and production output. The objective terms, constraints, grid, and replay settings are in `REPRODUCIBILITY.md`. Across seeds 42–44, this single controller made two policy-checked writes per seed, without rejection. It raised J from approximately 67.2 to $J/J^* = 0.965$–0.972, with $J^* = 89.894$ from an admissible setpoint grid. This finite restricted grid supplies a reference normalization, not a global continuous optimum. The result links the controller's native objective to admissible commands and endpoint evidence, exercising a conventional strategy through the same execution services.

E. Deployment Composition and Confirmation Semantics (Q5)

The deployment instance contained 34 provisioned lines, 111–113 channels, 67 online DAQ nodes, 77 write nodes, 52 recipes, and 385 bindings, with 17 lines running. These objects coexisted on one simulator-backed host. Nine pipeline executions passed 782 checks (Table III).

The injection benchmark followed observation, proposal, pacing gate, approval, batch dispatch, and recipe/readback/mirror comparison. The three sources agreed at the endpoint's recording resolution (maximum deviation 0.0 bar; tolerances 0.05 bar for device readback and 0.51 bar for the bucketed mirror). This verifies setpoint consistency rather than process convergence. In S4, the initial proposal was cadence-refused; the reproposal followed 69.014 s later, and observation-to-verification at 65 bar took 96.105 s. The archived verdict remained `converged=false`. S5 began 96.129 s after the S4 observation, applied a 40-to-40.5°C setpoint, refused a further write, waited 66.046 s, and restored 40°C. These are server/harness event spans, not process-convergence evidence. Across the campaign, approval-to-dispatch spans were 6–9 ms.

Seven fault tests exercised receipt grading. A mock acceptance required a reread; a broker outage failed one parameter while its healthy sibling verified; and a publish-only endpoint remained unverified after three attempts despite returning a broker receipt. The suite also tested readback-based promotion, confirmation of manual-to-automatic mode changes, and approval before manual-mode recipe dispatch. MES application-receipt semantics were covered by route tests; the campaign itself used MES only in read mode. These checks show that receipt grades followed the available endpoint evidence under the injected conditions.

F. Operational Recipe Workflow Case Study (Q6)

The October 18 recipe archive (R6 in the accompanying reproducibility notes) pairs recipe-path controls, receipts, and process observations. The replay entry is `scripts/e2e-injection-recipe-closedloop.mjs`. The setup used OPC UA setpoints and HTTP quality acquisition against the injection simulator. A live agent separately diagnosed insufficient packing, but the archived intervention sequence and approvals were scripted.

Starting from 45 bar and a fixed 6-s hold time, the script proposed 5-bar pressure increments. Each step called `recipe_trial`, answered the pending approval card, and retained the batch receipt. After 65 s, it queried platform control-node values and DAQ outputs. It stopped when weight was in $[32.15, 32.85]$ g, flash was at most 0.4%, and sink mark was at most 1.5%, or after at most six pressure proposals. The replay also defines overshoot compensation and a hold-time fallback. The archived successful run used four pressure proposals and no hold-time change. The harness set `AW_TRIAL_INTERVAL_MS=0`, disabling the per-line trial cadence, and issued `emergency=true` trials to bypass the separate recipe-operation interval. Approval, admission, and command confirmation remained active. This configuration tests the execution chain with experimental pacing. An earlier archived attempt exercised the default 300-s cadence: it refused a premature trial and the target was not attained in that attempt.

Pressure advanced $45 \rightarrow 50 \rightarrow 55 \rightarrow 60 \rightarrow 65$ bar; weight increased $31.28 \rightarrow 31.49 \rightarrow 31.74 \rightarrow 32.05 \rightarrow 32.30$ g, and sink mark decreased from 1.977% to 1.277%. Final flash was 0.019%. Each trial receipt reported one verified parameter out of one. The displayed controls are platform-node snapshots, while the receipt supplies the archived verification grade. After the final check, `recipe_update` saved version 2 with 65 bar/6 s. No `recipe_apply` call is archived. Q6 did not test applying the saved version. The recipe log contains four trials and the update, while the optimization journal is empty, as expected for this lifecycle.

At 60 bar, the 32.05-g observation remained below the 32.15-g acceptance limit; at 65 bar, all three quality criteria passed. Pressure was the sole changed control. The recipe-update rationale contains different quality figures without a linked raw sample, so the plot uses the archived final snapshot.

The quality assertion allowed a missing melt-temperature value, and the freshness check used the newest timestamp among available sensors. The reported pass therefore covers the three observed quality quantities; temperature and individual-sensor freshness remain outside that assertion. The additional archive is not an independent replication. A separate later check started at 65 bar, 32.29 g, and 1.385% sink mark; one 70-bar trial ended at 32.55 g and 0.976%. It is a new initial state and is not appended to the plotted trace.

October 23 boundary checks distinguish historical replay from current operation. The public interface refused all eight direct-node attempts (zero writes). A separate recipe check verified one 45-to-50-bar trial, then refused requests at 66 and 132 s under the default 300-s cadence. Restoration failed: pressure remained at 50 bar rather than the requested 45-bar

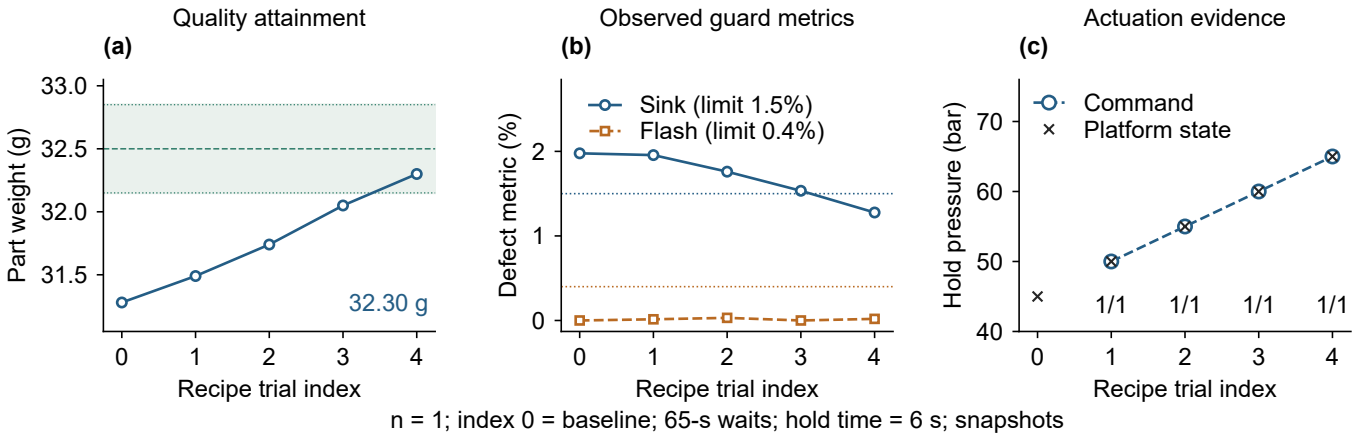


Fig. 5. October 18 recipe-path injection case study ($n = 1$ simulator trajectory). (a) Weight with the 32.15–32.85-g band and 32.5-g center. (b) Sink and flash snapshots with 1.5% and 0.4% limits. (c) Requested pressure and platform-node state after each 65-s wait; overlapping symbols indicate equality, and each trial receipt reports 1/1 parameter verified. Hold time remains 6 s. Points are discrete snapshots, not physical PLC traces; the harness supplied approvals.

baseline, and only 4/8 harness assertions passed. Final weight was 31.46 g; no recipe update was saved. These failed checks are not repetitions of the accelerated October 18 trajectory; their full outcomes are retained in the reproducibility notes.

V. Engineering Interpretation

Across four seed-42 simulator scenes, the historical archive shows that scene-specific signals, controls, and terminal objectives can be exercised through a common task and evidence structure. The protocol archive separately covers acquisition over five families and writes over four mappings; the deployment suite checks composition and confirmation behavior across nine sequential executions. Together, these archives support integration of the tested interfaces and workflows. They do not test how current channel profiles and per-agent bindings change tool availability or task outcomes across the four scenes: Q4 used a historical direct-node path, while the independent channel/profile checks used temporary storage and mock I/O rather than process-response trials.

The recipe case has a similarly specific scope. Q6 demonstrates an inspectable recipe workflow in one simulator trajectory, but the harness disabled the line cadence and used emergency trials to bypass the separate recipe-operation interval. The accepted definition was saved but not applied. A default-paced attempt did not reach its target, and later current- interface checks refused eight direct-node writes but exposed incomplete recipe restoration: pressure remained at 50 bar instead of the requested 45 bar, with only four of eight harness assertions passing. Q6 therefore does not establish default-paced recipe completion or recovery readiness.

The system's engineering contribution is the separation of collaboration, scene configuration, semantic device access, and route-specific execution records. This separation makes it possible to inspect a proposal, the resulting endpoint evidence, and the process observation as different parts of one task. It does not establish operator effectiveness: approval records do not measure comprehension or complacency [24]–[26]. Likewise, a restricted tool surface does not remove deployment security requirements [19], [27], [28], and the framework does not replace independently engineered PLC or safety-instrumented protection under the IEC 61511 process-safety lifecycle [29].

The next evaluation should use a versioned current build to vary profile and binding configurations across matched scenes and record their effect on tool availability and task outcomes, test recipe operation at default pacing including restoration, and compare task outcomes against declared baselines. Physical-plant validation is also needed before claiming transfer beyond the repository simulators; Tennessee Eastman offers one additional process setting [30]. Model-assisted control response remains unevaluated.

VI. Conclusion

This paper presents a configurable industrial-agent framework that connects collaborative harnesses, production context, process analysis, semantic industrial I/O, and distinct node and recipe workflows. Four configured simulator scenes passed their terminal checks, while separate protocol, deployment, live-engine, controller, and recipe archives exercised complementary parts of the implementation. The evidence supports integration of the tested interfaces and route behaviors; it does not yet validate current profile/binding adaptation across scenes, default-paced recipe recovery, physical transfer, or comparative control benefit. These results establish a basis for matched cross-scene and physical validation, rather than a claim of plant-level effectiveness.

References

[1] K.-C. Huang, H.-T. Lin, C.-H. Lin, and M.-T. Huang, "Large language model integration in industrial cyber-physical systems: A multi-modal architecture for automated control generation," in *Proc. Int. Conf. on Control, Automation and Diagnosis (ICCAD)*, 2026, pp. 1–6, doi: 10.1109/ICCAD69956.2026.11643124.

[2] K. Köhle, D. Anicic, T. A. Runkler, and R. Graf, "LLM-driven multi-agent control for skill-based smart manufacturing," in *Proc. IEEE 31st Int. Conf. on Emerging Technologies and Factory Automation (ETFA)*, 2026, pp. 1–8, doi: 10.1109/ETFA68377.2026.11725257.

[3] M. Waseem, K. Bhatta, C. Li, and Q. Chang, "Pretrained LLMs as real-time controllers for robot operated serial production line," *Mach. Learn.: Eng.*, vol. 2, no. 1, 2026, art. no. 015006; doi: 10.1088/3049-4761/ae4ee8.

[4] A. Biskupovic, A. Villalonga, F. Castaño, R. E. Haber, and F. Núñez, "Agent-based supervision for service-oriented industrial cyber–physical systems," *IEEE Trans. Ind. Inform.*, vol. 21, no. 3, pp. 2719–2728, 2025, doi: 10.1109/TII.2024.3514178.

[5] Z. Liu *et al.*, "Agents4PLC: Automating closed-loop PLC code generation and verification in industrial control systems using LLM-based agents," *IEEE Trans. Softw. Eng.*, vol. 52, no. 5, pp. 1672–1687, 2026, doi: 10.1109/TSE.2026.3667895.

[6] R. Wu, J. Ai, and T. Li, "InstructMPC: A human-LLM-in-the-loop framework for context-aware control," in *Proc. 2025 IEEE 64th Conference on Decision and Control (CDC)*, 2025, pp. 172–179, doi: 10.1109/CDC57313.2025.11312156.

[7] J. Lim and I. Kovalenko, "A large language model-enabled control architecture for dynamic resource capability exploration in multi-agent manufacturing systems," in *Proc. IEEE 21st Int. Conf. on Automation Science and Engineering (CASE)*, 2025, pp. 2088–2095, doi: 10.1109/CASE58245.2025.11163802.

[8] Z. Zhao *et al.*, "A large language model-based multi-agent manufacturing system for intelligent shopfloors," *Adv. Eng. Inform.*, vol. 69, 2026, art. no. 103888; doi: 10.1016/j.aei.2025.103888.

[9] M. S. Gill, J. Vyas, A. Markaj, F. Gehlhoff, and M. Mercangöz, "Leveraging LLM agents and digital twins for fault handling in process plants," in *Proc. IEEE ETFA*, 2025, pp. 1–8, doi: 10.1109/ETFA65518.2025.11205597.

[10] B. Hofmann, N. Piechulek, S. Kreitlein, J. Franke, and P. Bruendl, "Beyond touch-based human-machine interface: Control your machines in natural language by utilizing large language models and OPC UA," *arXiv preprint arXiv:2510.11300*, 2025. [Online]. Available: https://arxiv.org/abs/2510.11300v2

[11] X. Jiang *et al.*, "Agentic AI for safety-aware process monitoring and fault diagnosis: A review," *Processes*, vol. 14, no. 13, 2026, art. no. 2112; doi: 10.3390/pr14132112.

[12] Model Context Protocol contributors, "Model Context Protocol: Tools," Specification, version 2026-07-28, 2026, accessed: Oct. 20, 2026. [Online]. Available: https://modelcontextprotocol.io/specification/2026-07-28/server/tools

[13] ISA, "ISA-88.00.01-2010: Batch control—part 1: Models and terminology," International Standard, 2010.

[14] ——, "ANSI/ISA-95.00.01-2025 (IEC 62264-1 mod): Enterprise-control system integration—part 1: Models and terminology," International Standard, 2025.

[15] J. Moyne, E. del Castillo, and A. M. Hurwitz, Eds., *Run-to-Run Control in Semiconductor Manufacturing*, 1st ed. Boca Raton, FL: CRC Press, 2000.

[16] H. Wang, C. M. Poskitt, and J. Sun, "AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents," in *Proc. IEEE/ACM 48th Int. Conf. on Software Engineering (ICSE)*, 2026, pp. 2938–2950, doi: 10.1145/3744916.3764546.

[17] E. Debenedetti *et al.*, "Defeating prompt injections by design," 2025, arXiv preprint arXiv:2503.18813. [Online]. Available: https://arxiv.org/abs/2503.18813

[18] U.S. Food and Drug Administration, "Part 11, electronic records; electronic signatures—scope and application," Guidance for Industry, 2003.

[19] IEC, "IEC 62443-3-3:2013: Industrial communication networks—network and system security—part 3-3: System security requirements and security levels," Standard, 2013.

[20] IEEE, "IEEE std 1815-2012: Standard for electric power systems communications—distributed network protocol (DNP3)," Standard, 2012.

[21] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu, "Safe reinforcement learning via shielding," in *Proc. AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, 2018, pp. 2669–2678, doi: 10.1609/aaai.v32i1.11797.

[22] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada, "Control barrier function based quadratic programs for safety critical systems," *IEEE Trans. Autom. Control*, vol. 62, no. 8, pp. 3861–3876, 2017, doi: 10.1109/TAC.2016.2638961.

[23] L. Sha, "Using simplicity to control complexity," *IEEE Softw.*, vol. 18, no. 4, pp. 20–28, 2001, doi: 10.1109/MS.2001.936213.

[24] R. Parasuraman and D. H. Manzey, "Complacency and bias in human use of automation: An attentional integration," *Hum. Factors*, vol. 52, no. 3, pp. 381–410, 2010, doi: 10.1177/0018720810376055.

[25] R. Parasuraman, T. B. Sheridan, and C. D. Wickens, "A model for types and levels of human interaction with automation," *IEEE Trans. Syst., Man, Cybern., Part A*, vol. 30, no. 3, pp. 286–297, 2000, doi: 10.1109/3468.844354.

[26] L. Bainbridge, "Ironies of automation," *Automatica*, vol. 19, no. 6, pp. 775–779, 1983, doi: 10.1016/0005-1098(83)90046-8.

[27] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. 16th ACM Workshop on Artificial Intelligence and Security*, 2023, pp. 79–90, doi: 10.1145/3605764.3623985.

[28] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and benchmarking prompt injection attacks and defenses," in *Proc. 33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 1831–1847. [Online]. Available: https://www.usenix.org/conference/usenixsecurity24/presentation/liu-yupei

[29] IEC, "IEC 61511-1:2016: Functional safety—safety instrumented systems for the process industry sector—part 1: Framework, definitions, system, hardware and application programming requirements," International Standard, 2016.

[30] J. J. Downs and E. F. Vogel, "A plant-wide industrial process control problem," *Comput. Chem. Eng.*, vol. 17, no. 3, pp. 245–255, 1993, doi: 10.1016/0098-1354(93)80018-I.